МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

НГТУ НЭТИ | Факультет прикладной математики и информатики

Кафедра прикладной математики

Лабораторная работа № 2

по дисциплине «Методы оптимизации»

МЕТОДЫ СПУСКА (0-го, 1-го и 2-го ПОРЯДКА И ПЕРЕМЕННОЙ МЕТРИКИ)

Группа ПМ-31 МИЛЛЕР И.

Вариант 3 ПАЛИЙ Н.

РОДИОНОВ К.

Преподаватель ФИЛИППОВА ЕЛЕНА ВЛАДИМИРОВНА

Новосибирск, 2026

Цель работы

Ознакомиться с методами поиска минимума функции n переменных в оптимизационных задачах без ограничений [1, 5, 7, 8].

Задание

1. Реализовать следующие методы поиска экстремума функции (разного порядка):

Метод	Порядок	Уровень сложности
Наискорейшего спуска	1	2
Бройдена	Переменной метрики	4

Включить в реализуемый алгоритм собственную процедуру, реализующую одномерный поиск по направлению;

2. С использованием разработанных алгоритмов исследовать их на квадратичной функции $f(\bar{x}) = 100(x_2 - x_1)^2 + (1 - x_1)^2$, функции Розенброка $f(\bar{x}) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$ и на заданной в соответствии с вариантом тестовой функции, осуществляя спуск из различных исходных точек (не менее двух). Исследовать сходимость алгоритма, фиксируя точность определения минимума/максимума, количество итераций метода и количество вычислений функции в зависимости от задаваемой точности поиска. Результатом выполнения данного пункта должны быть выводы об объёме вычислений в зависимости от задаваемой точности и начального приближения;
3. Построить траекторию спуска различных алгоритмов из одной и той же исходной точки с одинаковой точностью. В отчёте наложить эту траекторию на рисунок с линиями разного уровня заданной функции;
4. Реализовать метод квадратичной интерполяции (метод парабол) для приближенного нахождения экстремума при одномерном поиске. Исследовать влияние точности одномерного поиска на общее число итераций и вычислений функции при разных методах одномерного поиска.

Функция, заданная по варианту:

$$f(x,y) = \exp\left\{-\left(\frac{x-3}{1}\right)^2 - \left(\frac{y-1}{3}\right)^2\right\} + 2\exp\left\{-\left(\frac{x-2}{2}\right)^2 - \left(\frac{y-2}{1}\right)^2\right\}$$

Исследование

Исследование с разной точностью поиска. Исследования будут проводиться на квадратичной функции, имеющей минимум в точке *(x, y) = (1, 1), f(x, y) = 0*.

- **При (x,y) = (5,4)**

Метод наискорейшего спуска

Точность	Итерации	Число выч. Функции	Найденная точка		Значение в точке
			x	y	
1e-03	7	586	1.00424307E+00	1.00426470E+00	1.80504498E-05
1e-04	8	672	1.00051063E+00	1.00038297E+00	1.89049649E-06
1e-05	9	754	1.00044439E+00	1.00044666E+00	1.98000096E-07
1e-06	10	840	1.00005495E+00	1.00004141E+00	2.13517595E-08
1e-07	11	922	1.00004792E+00	1.00004817E+00	2.30252323E-09

Метод Бройдена

Точность	Итерации	Число выч. Функции	Найденная точка		Значение в точке
			x	y	
1e-03	5	427	1.00044470E+00	1.00044697E+00	1.98271722E-07
1e-04	5	427	1.00044470E+00	1.00044697E+00	1.98271722E-07
1e-05	6	513	1.00004705E+00	1.00004729E+00	2.21935692E-09
1e-06	6	513	1.00004705E+00	1.00004729E+00	2.21935692E-09
1e-07	7	599	1.00000498E+00	1.00000500E+00	2.48411180E-11

- **При (x,y) = (9,8)**

Метод наискорейшего спуска

Точность	Итерации	Число выч. Функции	Найденная точка		Значение в точке
			x	y	
1e-03	11	922	1.01034234E+00	1.01039577E+00	1.07249496E-04
1e-04	12	1008	1.00308455E+00	1.00263737E+00	2.95119071E-05
1e-05	14	1176	1.00085245E+00	1.00072923E+00	2.24479341E-06
1e-06	16	1344	1.00023459E+00	1.00020058E+00	1.70724998E-07
1e-07	18	1512	1.00006480E+00	1.00005543E+00	1.29779390E-08

Метод Бройдена

Точность	Итерации	Число выч. Функции	Найденная точка		Значение в точке
			x	y	
1e-03	7	599	1.00284991E+00	1.00286464E+00	8.14369823E-06
1e-04	7	599	1.00284991E+00	1.00286464E+00	8.14369823E-06
1e-05	8	685	1.00078588E+00	1.00078994E+00	6.19251791E-07
1e-06	9	771	1.00021671E+00	1.00021783E+00	4.70881733E-08
1e-07	10	857	1.00005976E+00	1.00006007E+00	3.58061002E-09

Исследование методов на целевых функциях. Заданная точность: $\varepsilon = 10^{-3}$, $x_0 = (5, 4)$

Метод наискорейшего спуска. Квадратичная функция												
i	(x_i, y_i)		$f(x_i, y_i)$	Направление поиска		Экстремум	$\lvert x_i - x_{i-1}\rvert$	$\lvert y_i - y_{i-1}\rvert$	$\lvert f_i - f_{i-1}\rvert$	Угол	Градиент функции	
1	4.48114605E+00	4.49889803E+00	1.21498911E+01	-2.08000017E+02	2.00000017E+02	2.49448993E-03	-5.18853948E-01	4.98898027E-01	-1.03850109E+02	1.58842484E+00	3.41189654E+00	3.55039553E+00
2	1.43060436E+00	1.32452581E+00	1.31068601E+00	-3.41189654E+00	-3.55039553E+00	8.94089740E-01	-3.05054169E+00	-3.17437222E+00	-1.08392051E+01	3.08321936E+00	2.20769207E+01	-2.12157119E+01
3	1.37553370E+00	1.37744819E+00	1.41392088E-01	-2.20769207E+01	2.12157119E+01	2.49448993E-03	-5.50706564E-02	5.29223795E-02	-1.16929392E+00	1.58999102E+00	3.68170494E-01	3.82897131E-01
4	1.04519503E+00	1.03389613E+00	1.48090993E-02	-3.68170494E-01	-3.82897131E-01	8.97243757E-01	-3.30338677E-01	-3.43552060E-01	-1.26582989E-01	3.11655308E+00	2.35016988E+00	-2.25977962E+00
5	1.03933255E+00	1.03953313E+00	1.55107266E-03	-2.35016988E+00	2.25977962E+00	2.49448993E-03	-5.86247510E-03	5.63699751E-03	-1.32580266E-02	1.59030498E+00	3.85502194E-02	4.01150914E-02
6	1.00486530E+00	1.00366675E+00	1.67324979E-04	-3.85502194E-02	-4.01150914E-02	8.94086979E-01	-3.44672492E-02	-3.58663808E-02	-1.38374768E-03	3.12110576E+00	2.49442062E-01	-2.39711255E-01
7	1.00424307E+00	1.00426470E+00	1.80504498E-05	-2.49442062E-01	2.39711255E-01	2.49448993E-03	-6.22230711E-04	5.97957311E-04	-1.49274529E-04	1.59067610E+00	4.15997566E-03	4.32636964E-03

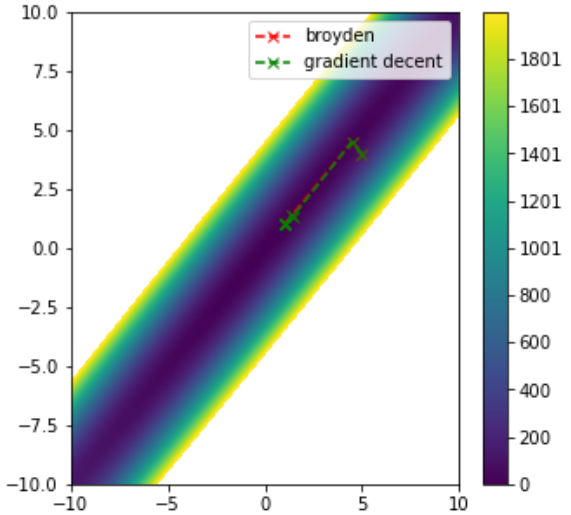
Метод Бройдена. Квадратичная функция												
i	(x_i, y_i)		$f(x_i, y_i)$	Направление поиска		Экстремум	$\lvert x_i - x_{i-1}\rvert$	$\lvert y_i - y_{i-1}\rvert$	$\lvert f_i - f_{i-1}\rvert$	Угол	Градиент функции	
											3.41189654E+00	3.55039553E+00
1	4.48114605E+00	4.49889803E+00	1.21498911E+01	-2.08000017E+02	2.00000017E+02	2.49448993E-03	-5.18853948E-01	4.98898027E-01	-1.03850109E+02	1.58842484E+00	[0.49889847	0.49889285]
											[0.49889285	0.50388721]
											3.68097386E-01	3.83030663E-01
2	1.37556388E+00	1.37747903E+00	1.41415007E-01	-3.47345690E+00	-3.49116969E+00	8.94089740E-01	-3.10558217E+00	-3.12141900E+00	-1.20084761E+01	3.13974506E+00	[0.50000003	0.50000003]
											[0.50000003	0.50500003]
											3.89378547E-02	4.05176682E-02
3	1.03972766E+00	1.03993025E+00	1.58239098E-03	-3.75564044E-01	-3.77479197E-01	8.94218245E-01	-3.35836220E-01	-3.37548785E-01	-1.39832616E-01	3.13914686E+00	[0.49999994	0.49999994]
											[0.49999994	0.50499994]
											4.11979507E-03	4.28694323E-03
4	1.00420327E+00	1.00422470E+00	1.77134070E-05	-3.97277564E-02	-3.99303447E-02	8.94195716E-01	-3.55243895E-02	-3.57055432E-02	-1.56467757E-03	3.13906011E+00	[0.49999996	0.49999996]
											[0.49999996	0.50499996]
											4.35956037E-04	4.53643637E-04
5	1.00044470E+00	1.00044697E+00	1.98271722E-07	-4.20336883E-03	-4.22480354E-03	8.94180159E-01	-3.75856900E-03	-3.77773550E-03	-1.75151353E-05	3.13905057E+00	[0.49999996	0.49999996]
											[0.49999996	0.50499996]

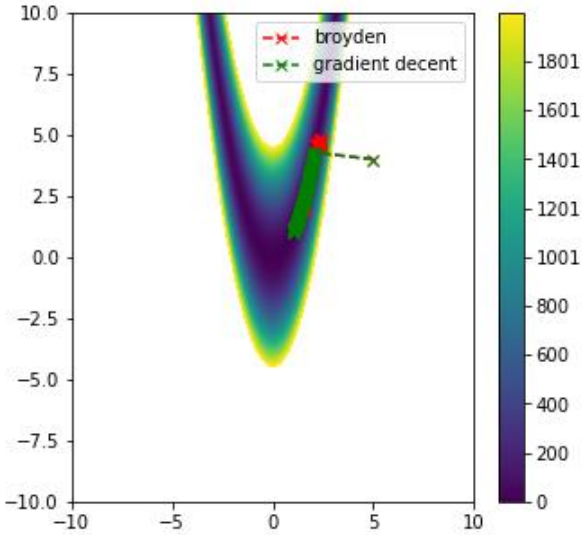
Метод наискорейшего спуска. Функция Розенброка												
i	(x_i, y_i)		$f(x_i, y_i)$	Направление поиска		Экстремум	$\lvert x_i - x_{i-1}\rvert$	$\lvert y_i - y_{i-1}\rvert$	$\lvert f_i - f_{i-1}\rvert$	Угол	Градиент функции	
1	2.07106851E+00	4.29283750E+00	1.14842167E+00	-4.20080105E+04	4.20000288E+03	6.97231659E-05	-2.92893149E+00	2.92837497E-01	-4.41148516E+04	1.92065371E+00	-7.67883090E-01	7.02541358E-01
2	2.07134423E+00	4.29258524E+00	1.14822719E+00	7.67883090E-01	-7.02541358E-01	3.59060426E-04	2.75716430E-04	-2.52254799E-04	-1.94483985E-04	1.86220405E+00	3.87581967E-01	4.23663993E-01

Метод Бройдена. Функция Розенброка												
i	(x_i, y_i)		$f(x_i, y_i)$	Направление поиска		Экстремум	$\lvert x_i - x_{i-1}\rvert$	$\lvert y_i - y_{i-1}\rvert$	$\lvert f_i - f_{i-1}\rvert$	Угол	Градиент функции	
											-7.67883090E-01	7.02541358E-01
1	2.07106851E+00	4.29283750E+00	1.14842167E+00	-4.20080105E+04	4.20000288E+03	6.97231659E-05	-2.92893149E+00	2.92837497E-01	-4.41148516E+04	1.92065371E+00	[0.00996922	0.09899902]
											[0.09899902	0.9901005]
											-6.59865496E-01	6.76459999E-01
2	2.07104629E+00	4.29261504E+00	1.14828415E+00	-6.18957098E-02	-6.19566883E-01	3.59060426E-04	-2.22242999E-05	-2.22461949E-04	-1.37522250E-04	2.79164055E+00	[0.00035012	0.00230215]
											[0.00230215	0.01806405]

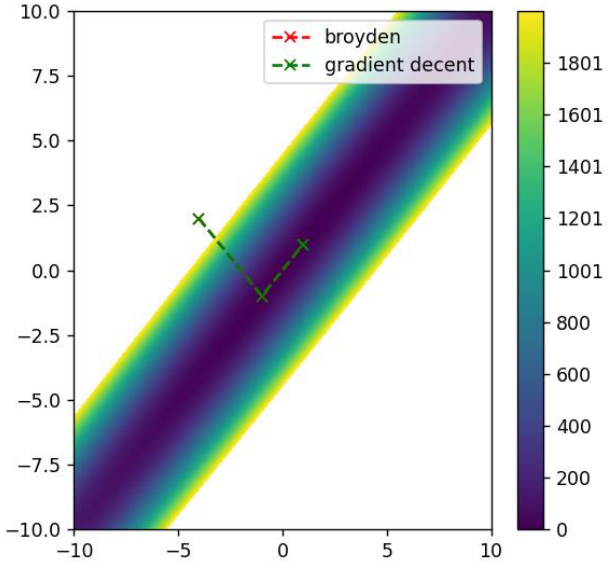
Траектории движения методов на разных функциях с разных точек

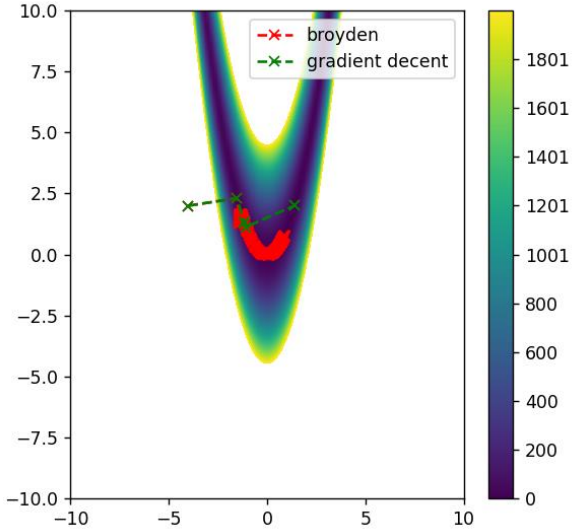
Начальная точка: [5, 4]





Начальная точка: [-4, 2]





Исследование зависимости количество вычислений от выбора одномерного метода поиска:

Метод наискорейшего спуска, квадратичная функция, $x_0 = (5, 4)$					
Одномерный метод	Точность ε	Число итераций	Число выч. функции	Найденная точка	Значение функции в точке
Метод парабол	10^{-7}	11	582	1.00004386e+00 1.00004409e+00	1.92901486e-09
Дихотомия	10^{-7}	11	1054	1.00004970e+00 1.00004996e+00	2.47693774e-09
Золотое сечение	10^{-7}	11	922	1.00004792e+00 1.00004817e+00	2.30252323e-09
Фибоначчи	10^{-7}	11	978	1.00004791e+00 1.00004815e+00	2.30113427e-09

Выводы:

При исследовании алгоритмов с разной точностью поиска получили, что число итераций тем меньше, чем меньше порядок задаваемой точности: во всех таблицах можно заметить, что с увеличением порядка точности увеличивается число итераций и, как следствие, число вычислений функции. Вдобавок чем меньше порядок задаваемой точности, тем меньше порядок определенного минимума/максимума: это можно увидеть по порядку значения функции в найденной точке.

Помимо этого получили, что число итераций (и число вычислений функции) зависят от начального приближения: чем дальше находится начальное приближение от точки экстремума, тем большее число итераций будет совершено при поиске. К тому же при большем расстоянии начального приближения от точки минимума/максимума, порядок полученного решения оказывается меньше.

При исследовании алгоритмов на целевых функциях получили, что на квадратичных функциях метод Бройдена сходится за меньшее число итераций, чем метод наискорейшего спуска. На функции Розенброка оба метода разошлись, — это может быть связано с порядком точности т.к. при выборе точности большего порядка метод показал удовлетворительные результаты.

При исследовании количества вычислений от выбора метода одномерного поиска получили, что для *квадратичной функции* наиболее оптимальным является метод парабол т.к. для получения решения он осуществил наименьшее число вычислений функции. Однако данный метод может легко разойтись, поэтому нужно следить за его сходимостью и накладывать ограничения в случае необходимости.

Код программы:

main.py

```
import numpy as np
import matplotlib.pyplot as plt
import one_dim_methods as odm
from math import acos
from math import exp

func_counter = 0

def reset_counter():
    global func_counter
    func_counter = 0


def quadratic_function(x, y):
    global func_counter
    func_counter += 1
    return 100 * (y-x) ** 2 + (1-x) ** 2


def rosenbrock_function(x, y):
    global func_counter
    func_counter += 1
    return 100 * (y - x ** 2) ** 2 + (1-x) ** 2


def personal_function(x, y):
    global func_counter
    func_counter += 1
    return -(np.exp(-((x-3) / 1 ) ** 2 - ((y-1) / 3) ** 2) + 2 * np.exp(-((x-2) /
2) ** 2 - ((y-2) / 1) ** 2))


def gradient(x, y, function):
    grad = np.zeros(2)
    grad[0] = odm.partial_derivative_x(x, y, function)
    grad[1] = odm.partial_derivative_y(x, y, function)
    return grad



def find_lamda(x, y, function, one_dim_method):
    lamda_interval = odm.lamda_interval_search(x, y, function)
    lamda = one_dim_method(x, y, function,
                           a=lamda_interval[0], b=lamda_interval[1])
    return lamda


def gradient_decent(function, one_dim_method=odm.golden_section_method, eps=1e-4):
    coords_prev = np.zeros(2)
```

```
    coords_next = np.zeros(2)
    delta_func = 1.0

    #coords_prev[0] = input("Enter x0 ")
    #coords_prev[1] = input("Enter y0 ")

    coords_prev[0] = -4.
    coords_prev[1] = 2.

    x_values = np.array(coords_prev[0])
    y_values = np.array(coords_prev[1])

    i = 0
    print("TABLE 2 FOR GRADIENT DECENT")
    print(f'iter num | '
          f'{"coords":^32} | '
          f'{"func_value":^15} | '
          f'{"direction":^32} | '
          f'{"lamda":^15} | '
          f'{"delta coords":^32} | '
          f'{"delta func":^15} | '
          f'{"angle":^15} | '
          f'{"gradient":^32} | ')

    while abs(delta_func) > eps:
        direction = -gradient(coords_prev[0], coords_prev[1], function)
        lamda = find_lamda(coords_prev[0], coords_prev[1],
                           function, one_dim_method)

        coords_next = coords_prev + lamda * direction

        func_value_next = function(coords_next[0], coords_next[1])
        func_value_prev = function(coords_prev[0], coords_prev[1])

        delta_coords = coords_next - coords_prev
        delta_func = func_value_next - func_value_prev

        grad_next = gradient(coords_next[0], coords_next[1], function)

        angle = acos(np.dot(coords_next, direction) /
                     np.linalg.norm(coords_next) / np.linalg.norm(direction))

        x_values = np.append(x_values, [coords_next[0]])
        y_values = np.append(y_values, [coords_next[1]])

        i += 1
        print(f'{i:^8} | '
              f'{coords_next[0]:^15.8e}  {coords_next[1]:^15.8e} | '
              f'{func_value_next:^15.8e} | '
              f'{direction[0]:^15.8e}  {direction[1]:^15.8e} | '
              f'{lamda:^15.8e} | '
```

```
                f'{delta_coords[0]:^15.8e}  {delta_coords[1]:^15.8e} | '
                f'{delta_func:^15.8e} | '
                f'{angle:^15.8e} | '
                f'{grad_next[0]:^15.8e}  {grad_next[1]:^15.8e} | ')

        coords_prev = coords_next.copy()

    print("\nINFO FOR TABLE1\n")
    print(f'eps   | iter_count | func_counter | {"coords":^30} | func value  ')
    print(f'{eps} | {i:^10} | {func_counter:^12} | {coords_next[0]:.8e}  '
          f'{coords_next[1]:.8e} | {func_value_next:.8e}')

    reset_counter()
    return x_values, y_values


def find_Hesse(delta_coords, delta_grad, Hesse):
    I = np.eye(2)
    denom  = np.dot(delta_grad, delta_coords)
    if(denom <= 1e-10):
        return Hesse
    coeff = 1.0 / denom
    A1 = I - coeff * np.outer(delta_coords, delta_grad)
    A2 = I - coeff * np.outer(delta_grad, delta_coords)
    Hesse = np.dot(A1, np.dot(Hesse, A2)) + coeff *
np.outer(delta_coords,delta_coords)
    return Hesse


def broyden_method(function, one_dim_method=odm.golden_section_method, eps=1e-4):
    coords_prev = np.zeros(2)
    coords_next = np.zeros(2)
    delta_func = 1.0

    I = np.eye(2)
    Hesse = I

    #coords_prev[0] = input("Enter x0 ")
    #coords_prev[1] = input("Enter y0 ")

    coords_prev[0] = -4.
    coords_prev[1] = 2.

    x_values = np.array(coords_prev[0])
    y_values = np.array(coords_prev[1])

    i = 0
    print("TABLE 2 FOR BROYDEN METHOD")
    print(f'{"Hesse":^25} | '
          f'iter num | '
          f'{"coords":^32} | '
```

```
        f'{"func_value":^15} | '
        f'{"direction":^32} | '
        f'{"lamda":^15} | '
        f'{"delta coords":^32} | '
        f'{"delta func":^15} | '
        f'{"angle":^15} | '
        f'{"gradient":^32} | ')

    while abs(delta_func) > eps:
        grad_prev = gradient(coords_prev[0], coords_prev[1], function)
        direction = -Hesse.dot(grad_prev)
        lamda = find_lamda(coords_prev[0], coords_prev[1],
                           function, one_dim_method)

        coords_next = coords_prev + lamda * direction
        grad_next = gradient(coords_next[0], coords_next[1], function)

        delta_coords = coords_next - coords_prev
        delta_grad = grad_next - grad_prev

        Hesse = find_Hesse(delta_coords, delta_grad, Hesse)

        func_value_next = function(coords_next[0], coords_next[1])
        func_value_prev = function(coords_prev[0], coords_prev[1])

        delta_func = func_value_next - func_value_prev

        angle = acos(np.dot(coords_next, direction) /
                     np.linalg.norm(coords_next) / np.linalg.norm(direction))

        x_values = np.append(x_values, [coords_next[0]])
        y_values = np.append(y_values, [coords_next[1]])

        i += 1
        print(f'{Hesse:} | '
              f'{i:^8} | '
              f'{coords_next[0]:^15.8e}  {coords_next[1]:^15.8e} | '
              f'{func_value_next:^15.8e} | '
              f'{direction[0]:^15.8e}  {direction[1]:^15.8e} | '
              f'{lamda:^15.8e} | '
              f'{delta_coords[0]:^15.8e}  {delta_coords[1]:^15.8e} | '
              f'{delta_func:^15.8e} | '
              f'{angle:^15.8e} | '
              f'{grad_next[0]:^15.8e}  {grad_next[1]:^15.8e} | ')

        coords_prev = coords_next.copy()

    print("\nINFO FOR TABLE1\n")
    print(f'eps    | iter_count | func_counter | {"coords":^30} | func value   ')
    print(f'{eps} | {i:^10} | {func_counter:^12} | {coords_next[0]:.8e}  '
          f'{coords_next[1]:.8e} | {func_value_next:.8e}')
```

```
    reset_counter()
    return x_values, y_values


def plot(function):
    x = np.linspace(-10, 10, 100)
    y = x

    xgrid, ygrid = np.meshgrid(x, y)
    zgrid = function(xgrid, ygrid)

    plt.figure(figsize=(5, 5))
    plt.contourf(xgrid, ygrid, zgrid, np.linspace(0, 2000, 2000))
    #plt.contourf(xgrid, ygrid, zgrid, np.linspace(-3, 2, 50))
    plt.colorbar()

    x_values_broyden, y_values_broyden = broyden_method(function)
    x_values_grad, y_values_grad = gradient_decent(function)

    plt.plot(x_values_broyden, y_values_broyden, 'x--r', label='broyden')
    plt.plot(x_values_grad, y_values_grad, 'x--g', label='gradient decent')

    plt.legend()
    plt.show()

if __name__ == "__main__":
    #plot(quadratic_function)
    plot(rosenbrock_function)
    # 0 2000 2000
    #plot(personal_function)
    # -3 2 50
```

one_dim_methods.py

```
import numpy as np
from math import sqrt
from math import log

def partial_derivative_x(x, y, function, delta_x=1e-9):
    return (function(x + delta_x, y) - function(x, y)) / delta_x

def partial_derivative_y(x, y, function, delta_y=1e-9):
    return (function(x, y + delta_y) - function(x, y)) / delta_y

def fibonacci_num(n):
    return (((1 + sqrt(5)) / 2) ** n - ((1 - sqrt(5)) / 2) ** n) / sqrt(5)

def find_n_for_fibonacci(*, a=-2, b=20, eps):
    t1 = log((1 + sqrt(5)) / 2)
    t2 = log(sqrt(5) * (b - a) * 4 / eps / ((1+sqrt(5)) ** 2))
    n = int(t2 / t1 + 1)
```

```
    return n

def lamda_interval_search(x, y, function):
    sigma = 0.0001
    lamda_prev = -1
    k = 1

    x_der = partial_derivative_x(x, y, function)
    y_der = partial_derivative_y(x, y, function)

    x_prev = x - lamda_prev * x_der
    y_prev = y - lamda_prev * y_der

    x_next = x - (lamda_prev + sigma) * x_der
    y_next = y - (lamda_prev + sigma) * y_der

    if function(x_prev, y_prev) > function(x_next, y_next):
        lamda_next = lamda_prev + sigma
        h = sigma
    else:
        lamda_next = lamda_prev - sigma
        h = -sigma

    flag = True

    while flag:
        h *= 2
        lamda_next += h

        x_prev, y_prev = x_next, y_next

        x_next = x - lamda_next * x_der
        y_next = y - lamda_next * y_der

        k += 1
        if function(x_prev, y_prev) > function(x_next, y_next):
            lamda_prev = lamda_next
        else:
            flag = False

    if lamda_prev < lamda_next:
        return np.array([lamda_prev-h/2, lamda_next])
    else:
        return np.array([lamda_next, lamda_prev+h/2])

def dichotomy(x, y, function, a=-2, b=20, eps=1e-7):
    a1 = a
    b1 = b

    x_der = partial_derivative_x(x, y, function)
    y_der = partial_derivative_y(x, y, function)
```

```
    i = 1
    while b1 - a1 > eps:
        delta = eps/10

        lamda1 = (a1 + b1 - delta) / 2
        lamda2 = (a1 + b1 + delta) / 2

        x1 = x - lamda1 * x_der
        y1 = y - lamda1 * y_der

        x2 = x - lamda2 * x_der
        y2 = y - lamda2 * y_der

        if function(x1, y1) < function(x2, y2):
            b1 = lamda2
        else:
            a1 = lamda1

        i += 1
    return (a1 + b1) / 2

def parabola_method(x, y, function, a=-2, b=20, eps=1e-7,  h=0.5):
    lamda_middle = (a + b) / 2
    lamda_min = - lamda_middle

    x_der = partial_derivative_x(x, y, function)
    y_der = partial_derivative_y(x, y, function)

    while abs(lamda_min - lamda_middle) > eps:
        lamda_left = lamda_middle - h
        lamda_right = lamda_middle + h

        x_left = x - lamda_left * x_der
        y_left = y - lamda_left * y_der

        x_right = x - lamda_right * x_der
        y_right = y - lamda_right * y_der

        x_middle = x - lamda_middle * x_der
        y_middle = y - lamda_middle * y_der

        lamda_min = 0.5 * ((function(x_left, y_left) *
                            (lamda_right + lamda_middle) - 2 *
                            function(x_middle, y_middle) *
                            (lamda_right + lamda_left) +
                            function(x_right, y_right) *
                            (lamda_middle + lamda_left)) /
                           (function(x_left, y_left) - 2 *
                            function(x_middle, y_middle) +
                            function(x_right, y_right)))
```

```
        h = abs(lamda_min - lamda_middle)
        lamda_middle = lamda_min
    return lamda_min

def golden_section_method(x, y, function, a=-2, b=20, eps=1e-7):
    a1 = a
    b1 = b

    coeff_1 = (3 - sqrt(5)) / 2
    coeff_2 = (sqrt(5) - 1) / 2

    lamda1 = a + coeff_1 * (b - a)
    lamda2 = a + coeff_2 * (b - a)

    x_der = partial_derivative_x(x, y, function)
    y_der = partial_derivative_y(x, y, function)

    x1 = x - lamda1 * x_der
    y1 = y - lamda1 * y_der

    x2 = x - lamda2 * x_der
    y2 = y - lamda2 * y_der

    func_value_1 = function(x1, y1)
    func_value_2 = function(x2, y2)

    i = 0
    while b1 - a1 > eps:
        a2 = a1
        b2 = b1

        if func_value_1 < func_value_2:
            b2 = lamda2
            lamda2 = lamda1
            x2 = x1
            y2 = y1
            func_value_2 = func_value_1
            lamda1 = a2 + coeff_1 * (b2 - a2)
            x1 = x - lamda1 * x_der
            y1 = y - lamda1 * y_der
            func_value_1 = function(x1, y1)
        else:
            a2 = lamda1
            lamda1 = lamda2
            x1 = x2
            y1 = y2
            func_value_1 = func_value_2
            lamda2 = a2 + coeff_2 * (b2 - a2)
            x2 = x - lamda2 * x_der
            y2 = y - lamda2 * y_der
```

```
            func_value_2 = function(x2, y2)

        b1 = b2
        a1 = a2
        i += 1
    return (a1 + b1) / 2

def fibonacci_method(x, y, function, a=-2, b=20, eps=1e-7):
    initial_length = b - a
    a1 = a
    b1 = b

    n = find_n_for_fibonacci(eps=1e-7)

    Fn2 = fibonacci_num(n + 2)

    lamda1 = a + fibonacci_num(n) / Fn2 * initial_length
    lamda2 = a + fibonacci_num(n + 1) / Fn2 * initial_length

    x_der = partial_derivative_x(x, y, function)
    y_der = partial_derivative_y(x, y, function)

    x1 = x - lamda1 * x_der
    y1 = y - lamda1 * y_der

    x2 = x - lamda2 * x_der
    y2 = y - lamda2 * y_der

    func_value_1 = function(x1, y1)
    func_value_2 = function(x2, y2)

    i = 1
    while i <= n:
        b2 = b1
        a2 = a1

        if func_value_1 < func_value_2:
            b2 = lamda2
            lamda2 = lamda1
            x2 = x1
            y2 = y1
            func_value_2 = func_value_1
            lamda1 = a2 + fibonacci_num(n - i) / Fn2 * initial_length
            x1 = x - lamda1 * x_der
            y1 = y - lamda1 * y_der
            func_value_1 = function(x1, y1)
        else:
            a2 = lamda1
            lamda1 = lamda2
            func_value_1 = func_value_2
            lamda2 = a2 + fibonacci_num(n - i + 1) / Fn2 * initial_length
```

```
            x2 = x - lamda2 * x_der
            y2 = y - lamda2 * y_der
            func_value_2 = function(x2, y2)

        b1 = b2
        a1 = a2
        i += 1
    return (a1 + b1) / 2
```